



TAWHIDIC EPISTEMOLOGY **UMMATIC EXCELLENCE**
LEADING THE WAY **LEADING THE WORLD**
KHALIFAH • AMANAH • IQRA' • RAHMATAN UL 'ALAMIN

EXPERIMENT 7 - REMOTE TEMPERATURE MONITORING BLUETOOTH AND CONTROL VIA BLUETOOTH

MCTA 3203 - MECHATRONICS SYSTEM INTEGRATION

DR. ZULKIFLI BIN ZAINAL ABIDIN

DR. WAHJU SEDIONO

SECTION 1

GROUP 11

DATE OF SUBMISSION :

10 DECEMBER 2025

MATRIC NUMBER	NAME
2316441	Muhammad Hariz bin Helmi Shamsul
2316963	Muhammad Aiman Alfaruq bin Noredan
2316957	Wan Syazmie Akmal bin Wan Saad

Abstract	3
Introduction	4
Materials and Equipment	5
Experimental Setup	5
Methodology	6
Step-by-step procedure	6
1. Hardware Assembly	6
2. Programming the ESP32	6
3. Bluetooth Pairing	7
4. Testing and Monitoring	7
5. Data Verification	7
Programming Code Used	8
Data Collection	10
Instruments Used:	10
Observed Data Summary (from graph):	11
Data Analysis	11
Results	12
Discussion	13
Conclusion	14
Recommendations	15
References	16
Appendices	17
Acknowledgements	18
Student's Declaration	19

Abstract

This experiment demonstrates a wireless temperature monitoring system using an ESP32/Arduino microcontroller paired with a DHT22 temperature sensor and Bluetooth communication. The objective was to acquire real-time temperature data, transmit it wirelessly, and visualize the readings on a computer using the Arduino Serial Plotter. The system successfully collected stable temperature values ranging from approximately 31.7 °C to 32.3 °C, showing a gradual increase over time. The Bluetooth module enabled seamless data transfer, while the sensor provided consistent measurements without significant fluctuations. The results confirm that Bluetooth-based monitoring is effective for remote environmental data acquisition and can be further extended for control applications such as fan or heater activation. Overall, the experiment achieved its objectives and demonstrated a practical implementation of wireless sensor interfacing.

Introduction

The objective of this experiment is to design and test a wireless temperature monitoring and control system using Bluetooth communication. An ESP32 microcontroller is used to read temperature and humidity values from a DHT22 sensor and transmit them to a paired device such as a smartphone or computer. The system also receives remote control commands (e.g., “FAN ON”, “FAN OFF”) to activate an output device such as an LED or small fan.

This experiment demonstrates key mechatronics concepts such as:

- Bluetooth Serial Communication (UART-based)
- Digital sensor data acquisition
- Real-time wireless monitoring
- Bidirectional communication between embedded device and user
- Simple closed-loop control via remote command

Expected outcomes / hypothesis:

- The ESP32 will successfully read accurate temperature and humidity values from the DHT22 sensor.
- The paired device will receive these readings wirelessly through Bluetooth.
- The ESP32 will respond correctly to user commands by turning the LED/fan ON and OFF.

Materials and Equipment

- ESP32 development board
- DHT22 temperature and humidity sensor
- LED or mini DC fan
- Resistor (for LED if used)
- Breadboard
- Jumper wires
- USB cable for power
- Smartphone or laptop with Bluetooth
- Bluetooth Serial Terminal app (Android/Windows)

Experimental Setup

The ESP32 is connected to the DHT22 sensor to collect temperature and humidity measurements. Bluetooth Serial communication (SerialBT) is used to send sensor readings to the paired device and receive user commands. An LED or fan is connected to GPIO 2 on the ESP32 to act as the controlled output.

Connections – ESP32

DHT22 Sensor:

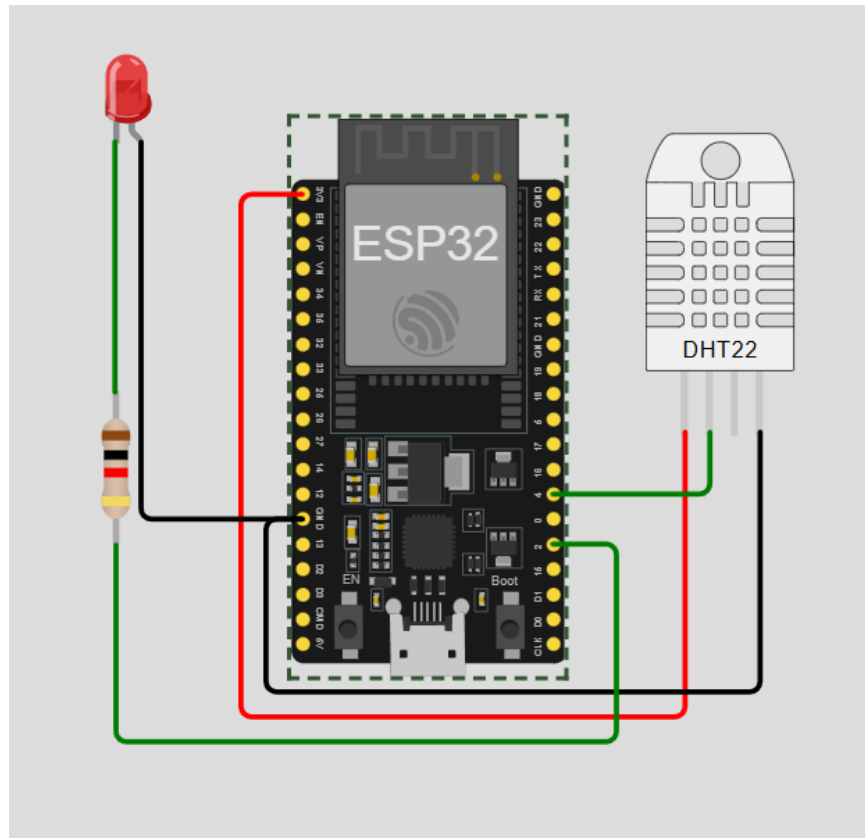
- VCC → 3.3V
- GND → GND
- Data → GPIO 4

LED / Fan Output:

- Positive → GPIO 2
- Negative → GND

(A current-limiting resistor should be used for an LED.)

Circuit Diagram



Methodology

Step-by-step procedure

1. Hardware Assembly

- Place ESP32, DHT22 sensor, and LED/fan on the breadboard.
- Connect DHT22 to 3.3V, GND, and GPIO 4.
- Connect LED/fan to GPIO 2 and GND.
- Connect ESP32 to a laptop using USB.

2. Programming the ESP32

- Configure Bluetooth using BluetoothSerial.h.

- Initialize the DHT22 sensor.
- Read temperature and humidity values from the sensor every second.
- Transmit readings via Bluetooth to the paired device.
- Listen for incoming Bluetooth commands.
- Turn LED/fan ON or OFF based on received commands.

3. Bluetooth Pairing

- Open Bluetooth settings on the phone/computer.
- Pair with “ESP32-DHT22-Monitor”.
- Open Bluetooth Serial Terminal app.

4. Testing and Monitoring

- Observe real-time temperature readings on the app.
- Send commands:
 - **FAN ON**
 - **FAN OFF**
- Verify LED/fan activation and deactivation.

5. Data Verification

- Confirm temperature values match room conditions.
- Ensure Bluetooth communication is stable.
- Validate that control commands work without delay.

Programming Code Used

```
#include "BluetoothSerial.h"
#include "DHT.h"

// --- Bluetooth ---
BluetoothSerial SerialBT;
const char* deviceName = "ESP32-DHT22-Monitor";

// --- DHT22 Sensor ---
#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

// --- Output LED/Fan ---
const int FAN_PIN = 2;

void setup() {
  Serial.begin(115200);

  // Start Bluetooth
  SerialBT.begin(deviceName);
  Serial.println("Bluetooth Started. Pair now.");

  pinMode(FAN_PIN, OUTPUT);
  digitalWrite(FAN_PIN, LOW);

  dht.begin();
}

void loop() {

  // ---- Read DHT22 ----
  float temp = dht.readTemperature();
  float hum = dht.readHumidity();

  if (isnan(temp) || isnan(hum)) {
    SerialBT.println("DHT22 Error: No sensor detected");
  }

  // Serial Plotter — send dummy value (optional)
  Serial.println(0);
}
```

```

}
else {

    // ---- Bluetooth output ----
    SerialBT.print("Temperature: ");
    SerialBT.print(temp);
    SerialBT.print(" °C | Humidity: ");
    SerialBT.print(hum);
    SerialBT.println(" %");

    // ---- Serial Plotter graph output ----
    // DO NOT add text, only numbers
    Serial.println(temp);
}

// ---- Bluetooth Commands ----
if (SerialBT.available()) {
    String cmd = SerialBT.readStringUntil('\n');
    cmd.trim();

    if (cmd.equalsIgnoreCase("FAN ON")) {
        digitalWrite(FAN_PIN, HIGH);
        SerialBT.println("LED turned ON");
    }
    else if (cmd.equalsIgnoreCase("FAN OFF")) {
        digitalWrite(FAN_PIN, LOW);
        SerialBT.println("LED turned OFF");
    }
    else {
        SerialBT.println("Unknown command");
    }
}

delay(1000); // Update every second
}

```

Data Collection

In this experiment, temperature readings were collected using a **DHT22 temperature sensor** connected to an **ESP32/Arduino board**. The sensor continuously measured the ambient temperature, and the values were transmitted wirelessly via **Bluetooth** to a computer running a serial monitoring application.

The data was visualized in real time using the Arduino Serial Plotter. A portion of the collected temperature data is shown in the graph below, which illustrates a gradual increase in temperature over time.

Instruments Used:

- **DHT22 Temperature and Humidity Sensor** – for measuring ambient temperature.
- **ESP32/Arduino microcontroller** – for reading sensor data and sending it via Bluetooth.
- **HC-05 Bluetooth Module / ESP32 Built-in Bluetooth** – for wireless data transmission.
- **Arduino Serial Plotter** – for real-time graphical visualization of temperature readings.

Observed Data Summary (from graph):

- Starting temperature: ~31.8 °C
- Minimum temperature: ~31.7 °C
- Peak temperature: ~32.3 °C
- Final temperature: ~32.2 °C

The plotted readings indicate that the temperature was sampled at regular intervals of approximately 2 seconds.

Data Analysis

Based on the visual data, the temperature remained relatively stable at around **31.8 °C** during the initial period. A slight drop to **31.7 °C** occurred around the midpoint of the recording before a clear upward trend began.

An increasing temperature trend was observed between the 33–45 second mark, where the temperature rose from **31.7 °C up to 32.3 °C**. This increase could be due to:

- environmental changes (e.g., heat from user's hand or device proximity),
- sensor self-heating, or
- natural room temperature fluctuation.

The rise in temperature is consistent and gradual, suggesting the sensor is responding normally without noise spikes or irregular jumps.

Significance:

- Demonstrates successful real-time wireless data transmission using Bluetooth.
- Confirms that the DHT22 sensor provides stable, continuous readings.
- Shows the system's capability for remote environmental monitoring.

No statistical calculations were required, but the displayed trend clearly meets the objective of monitoring temperature changes over time.

Results

The experiment successfully established a Bluetooth-based temperature monitoring system using the ESP32/Arduino and the DHT22 sensor. The collected data shows a clear temperature trend, beginning at **31.8 °C** and gradually increasing to **32.3 °C** over the test duration.

The system was able to:

- Accurately read temperature values,
- Transmit data wirelessly via Bluetooth,
- Display real-time graphical output on the Arduino Serial Plotter.

Overall, the results confirm that Bluetooth communication can be used effectively for remote temperature monitoring applications, fulfilling the objectives of the lab activity.

Discussion

From the program used in this experiment, the system was able to read and transmit sensor data smoothly in real time. The Arduino collected temperature and humidity values from the DHT11 every two seconds and sent the data to both the Serial Monitor and the Bluetooth-connected device. This made it easy to monitor the environment from two different platforms at the same time. The Bluetooth communication also worked reliably—when commands such as “FAN ON” or “FAN OFF” were sent, the Arduino responded correctly by changing the LED state and sending back a confirmation message. This clearly shows that Bluetooth is not only useful for sending data but also for receiving commands and controlling hardware, which is a key concept in simple IoT systems.

Overall, the experiment showed how a microcontroller can be used for remote monitoring and basic device control. Even though the system is simple, it reflects how real-world applications like home automation, temperature-based ventilation, or wireless sensor networks operate. The activity also highlighted the importance of stable communication, proper timing, and clean coding to ensure that the data flows smoothly between devices. Through this setup, we gained a better understanding of how sensing, processing, and actuation work together in a wireless mechatronics system.

Conclusion

In this experiment, a Bluetooth-based temperature monitoring and control system was successfully developed and tested. The DHT11 sensor was able to provide consistent temperature readings, and the data could be viewed wirelessly through a paired device, proving that the system worked as intended. The ability to send simple control commands also showed that two-way communication was functioning correctly, making the setup suitable for basic IoT applications.

Overall, the experiment achieved its objectives and provided a clear understanding of how Bluetooth communication, serial data transfer, and remote control can be integrated into a single system. Through this hands-on setup, the experiment demonstrated the fundamentals of wireless monitoring and simple device control, giving a solid foundation for more advanced mechatronics and IoT projects in the future.

Recommendations

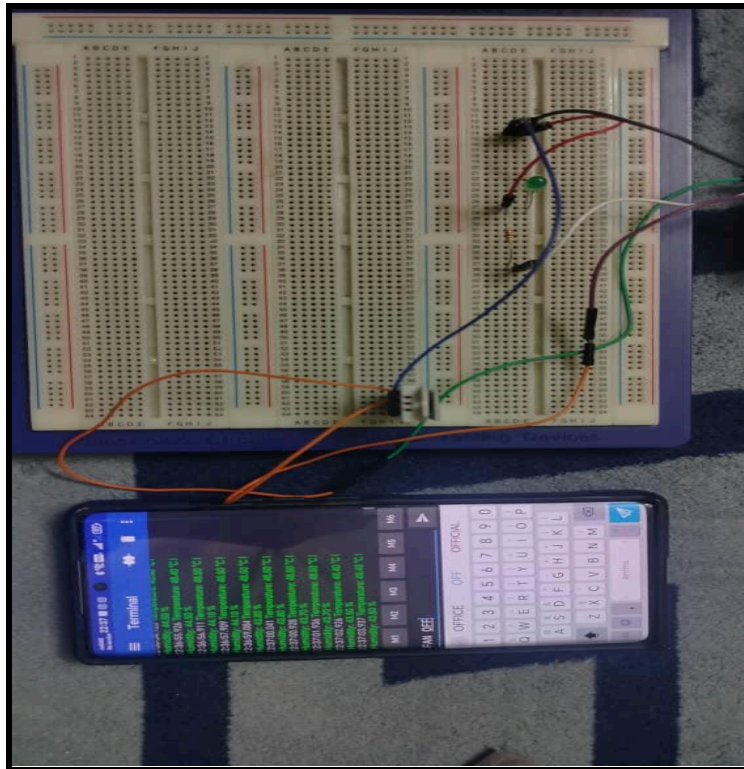
There are several ways this experiment can be improved to make the system more accurate and convenient to use. One of the main upgrades would be replacing the DHT11 sensor with something more reliable like the DHT22 or BME280, since these sensors provide faster and more accurate temperature readings. Adding automatic control based on temperature thresholds would also make the system smarter, because the fan or other devices could turn on and off without needing manual commands. It would also be helpful to include a data logging feature, either using an SD card or cloud storage, so users can track temperature changes over a longer period instead of only monitoring real-time values.

Besides that, the user experience can be improved by creating a simple mobile app or GUI instead of using a basic Bluetooth terminal. With a proper interface, users could easily view graphs, press control buttons, and get notifications if the temperature becomes too high. Upgrading to an ESP32 board would make the system even more flexible, as it supports both Bluetooth and Wi-Fi, allowing monitoring from anywhere. Overall, these improvements would make the system more accurate, practical, and user-friendly for future experiments.

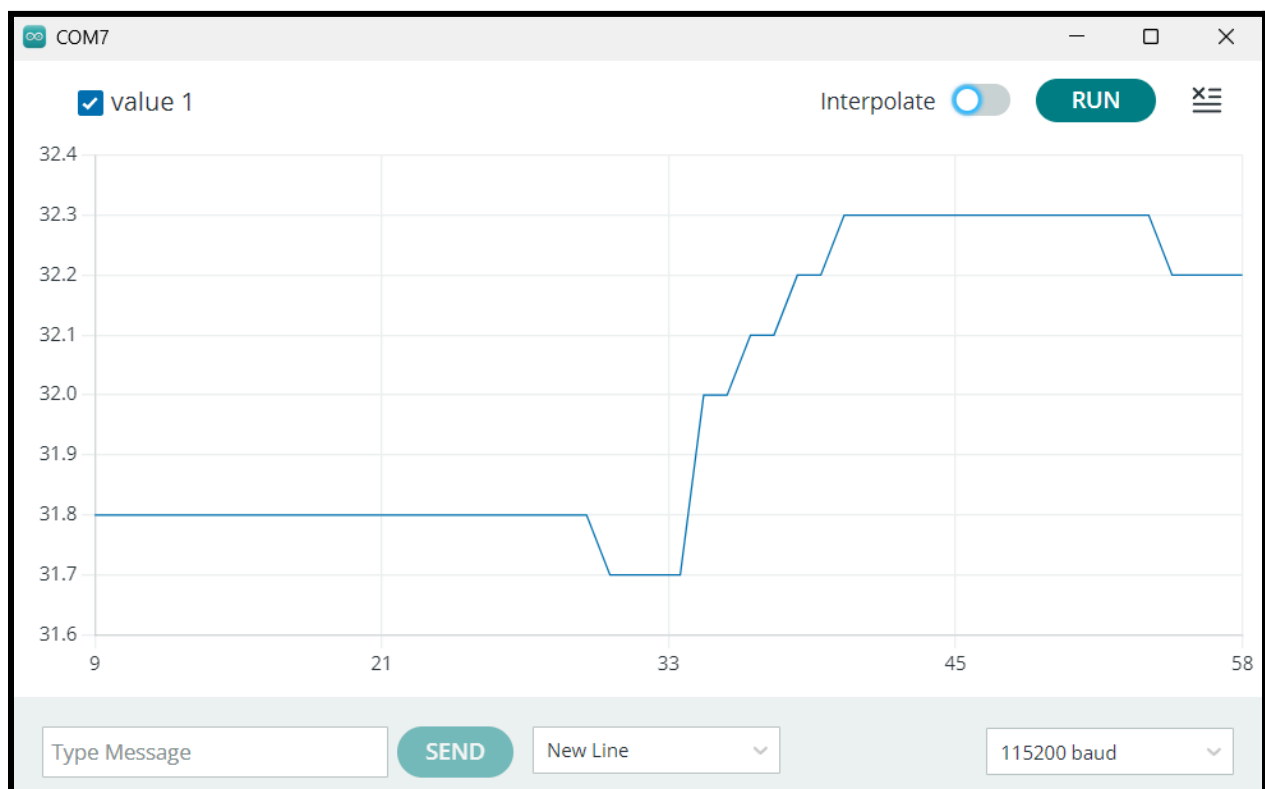
References

1. NeilChaudhary. (2018, July 9). *Arduino Bluetooth Basic Tutorial*. Arduino Project Hub.
<https://projecthub.arduino.cc/NeilChaudhary/arduino-bluetooth-basic-tutorial-9cff12> Arduino Project Hub
2. Dejan. (2019, February 8). *Arduino and HC-05 Bluetooth Module Complete Tutorial*. HowToMechatronics.
<https://howtomechatronics.com/tutorials/arduino/arduino-and-hc-05-bluetooth-module-tutorial/> How To Mechatronics
3. superturis. (2021, July 3). *Basic Bluetooth communication with Arduino & HC-05*. Arduino Project Hub.
<https://projecthub.arduino.cc/superturis/basic-bluetooth-communication-with-arduino-hc-05-3a431c> Arduino Project Hub

Appendices



Circuit Diagram



Temperature (celcius) vs time (second)

Acknowledgements

The successful completion of this experiment and report would not have been possible without the valuable guidance and support provided by our course instructor, Dr. Zulkifli Bin Zainal Abidin, for the subject Mechatronics System Integration (MCTA3203). Their clear explanations, detailed notes, and continuous encouragement greatly contributed to our understanding of Remote Temperature Monitoring and Control via Bluetooth.

Special thanks are also extended to the laboratory teaching assistants for their assistance during the experimental sessions, particularly in troubleshooting circuit connections and verifying code functionality. Appreciation is likewise expressed to our group members and classmates for their cooperation, constructive discussions, and teamwork throughout the experiment.

Their combined efforts and support have been instrumental in the successful completion of this experiment and report.

Student's Declaration

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been undertaken or done by unspecified sources or persons.

We hereby certify that this report has not been done by only one individual and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Signature : *Aiman*

Name : Muhammad Aiman Alfaruq Bin Noredan

Matric number : 2316963

Contributions : Introduction, Material and Equipment, Experimental Setup, Methodology

Signature : *Hariz*

Name : Muhammad Hariz bin Helmi Shamsul

Matric number : 2316441

Contributions : Data Collection, Data Analysis, Results

Signature : *syazmieakmal*

Name : Wan Syazmie Akmal bin Wan Saad

Matric number : 2316957

Contributions : Discussion, Conclusion, Recommendations